**UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA**
ESCUELA DE POSGRADO




# Trabajo Final - Parte 1

**AUTOR(ES)**
Sebastian Tadeo Gallegos Mostajo
Carlos Kiyoshi Fukumoto Chávez
Axel Daniel Fuentes Pacheco
André Welky Arbaiza Abanto
Gonzalo Fausto Chu Siu Ting

**DOCENTE**
Luis Chicoma Chávez

**CURSO**
Base de datos

Lima – Perú
2026

# Índice

**1. Análisis del negocio, modelo relacional e implementación** **2**
1.1. Contexto y modelo de negocio . . . 2
1.2. Procedencia y alcance de los datos . . . 2
1.3. Modelo entidad–relación . . . 2
1.4. Claves, relaciones y decisiones de diseño . . . 3
1.5. Implementación en PostgreSQL . . . 3

**2. Consultas SQL orientadas al negocio** **3**
2.1. Resultados e interpretación . . . 4
2.2. Consultas ejecutables . . . 5

**3. Feature Engineering mediante SQL** **5**
3.1. Objetivo y unidad de análisis . . . 5
3.2. Variables y justificación . . . 5
3.3. Construcción del conjunto analítico . . . 6
3.4. Resultados observados . . . 6
3.5. Consideraciones para entrenamiento futuro . . . 6

**4. Optimización de consultas e índices** **6**
4.1. Diseño de la comparación . . . 6
4.2. Índices implementados . . . 7
4.3. Comparación de tiempos . . . 7
4.4. Comparación de planes: costos, filas y tiempos . . . 7
4.5. Planificación y cuellos de botella . . . 8
4.6. Alcance y reproducibilidad . . . 8

**5. Conclusiones** **8**

**6. Referencias y archivos fuente** **9**

**A. Código SQL: consultas de negocio (Parte 2)** **9**

**B. Código SQL: Feature Engineering (Parte 3)** **14**

**C. Código SQL: análisis de planes e índices (Parte 4)** **18**

# 1 Análisis del negocio, modelo relacional e implementación

## 1.1 Contexto y modelo de negocio

El caso de estudio representa una plataforma de comercio electrónico multicategoría. La actividad comercial comprende la exposición de productos al público, la exploración del catálogo, la incorporación de artículos al carrito y el registro de compras. El proyecto analiza trazas de interacción del cliente, no la operación completa de una compañía real identificada.

Los usuarios principales son visitantes o clientes representados por un identificador `user_id`. La plataforma ofrece bienes de varias categorías, entre ellas teléfonos inteligentes, televisores, accesorios electrónicos, equipos informáticos y herramientas. El canal observable en el conjunto de datos es el canal digital; no se dispone de evidencia para diferenciar tráfico web de aplicaciones móviles, puesto que la tabla de dispositivos está vacía.

El proceso seleccionado es el recorrido de navegación a compra. Participan el usuario de la tienda y el sistema de registro transaccional. Los eventos centrales son `view` (visualización), `cart` (incorporación al carrito) y `purchase` (compra). Cada registro conserva marca temporal, tipo de evento, producto, categoría, marca, precio, usuario y sesión. Con ellos se pueden responder preguntas sobre productos populares, ingresos por categoría, conversión, abandono de carrito, trayectoria temporal y propensión de compra.

## 1.2 Procedencia y alcance de los datos

El repositorio indica que el origen es el conjunto público *eCommerce behavior data from multi category store*, disponible en Kaggle. La base proporcionada es un extracto del 1 de noviembre de 2019, aproximadamente de 00:00 a 00:31, con **4,083 eventos**, de los cuales **51** corresponden a compras, concentradas en **39 sesiones compradoras**. El conjunto representa una ventana acotada y no permite extrapolar patrones diarios o mensuales. No se ha confirmado la divisa del precio: los importes se presentan en unidades monetarias del dataset, sin asignarles una moneda específica.

## 1.3 Modelo entidad–relación

La implementación se encuentra bajo el esquema PostgreSQL `analisis_ecommerce_produccion`. Sus tablas principales son las siguientes:

| Entidad | Propósito y atributos relevantes |
|---|---|
| `usuarios` | Usuarios identificados por `user_id`; nombre y fecha de registro opcionales. |
| `sesiones` | Sesiones identificadas por `user_session`, relacionadas con usuario y fecha de inicio. |
| `eventos` | Hechos de navegación y compra; identificador de evento, instante, tipo, producto, categoría, marca, precio, usuario y sesión. |
| `productos` | Catálogo de productos con `product_id`, marca y precio estándar. |
| `marcas` | Catálogo de marcas, con identificador y nombre. |
| `categorias` | Categorías con identificador, código y nombre. |
| `tipos_evento` | Catálogo de tipos de evento. |
| `dispositivos` | Identificador de dispositivo, sesión, tipo de dispositivo y dirección IP. |
| `temp` | Tabla auxiliar de carga con los campos originales del evento. |

Tabla 1: Tablas identificadas en el respaldo entregado.



Figura 1: Diagrama del modelo compartido por el equipo.

### 1.4 Claves, relaciones y decisiones de diseño

Los identificadores `event_id`, `product_id`, `user_id`, `user_session`, `brand_id` y `category_id` permiten relacionar hechos y dimensiones. En particular, un usuario puede generar múltiples sesiones; cada sesión puede registrar varios eventos; un producto puede aparecer en muchos eventos y una marca puede agrupar múltiples productos. Se utilizan relaciones 1:N materializadas mediante claves foráneas, sin necesitar una tabla intermedia N:M para esos vínculos. En el diseño, los eventos actúan como registros transaccionales y sus dimensiones aportan información descriptiva.

La tabla `eventos` conserva también `category_code` y `brand`, aun cuando existen tablas de categorías y marcas. Esta redundancia se interpreta como conservación del valor observado al momento del evento y facilita reconstruir el estado histórico; no debe presentarse como una normalización estricta a 3FN sin justificación. La tabla `temp` es una estructura de carga, no una entidad de negocio normalizada. Para un cumplimiento estricto de 3FN, correspondería evaluar la separación de atributos repetidos y utilizar referencias consistentes a los catálogos; el repositorio entregado no documenta una demostración formal de 3FN.

### 1.5 Implementación en PostgreSQL

El respaldo `Trabajo_Final.sql` contiene la instrucción `CREATE SCHEMA`, definiciones `CREATE TABLE`, secuencias, comprobaciones de precios no negativos y carga de registros mediante `COPY`. Las tablas presentan identificadores primarios y vínculos foráneos del diseño. Los tipos de datos incluyen `BIGINT`, `INTEGER`, `VARCHAR`, `TIMESTAMP` y `NUMERIC(10,2)`; por ejemplo, los precios no permiten valores negativos. La carga del respaldo se realiza con `COPY`, aunque la rúbrica exige específicamente evidenciar `INSERT INTO`; por lo tanto, conviene incluir al menos una operación `INSERT INTO` verificable en el script definitivo. Asimismo, se debe documentar de manera explícita dónde se usan `UNIQUE`, `CHECK` y `NOT NULL`.

## 2 Consultas SQL orientadas al negocio

El equipo desarrolló diez consultas SQL: cinco básicas o intermedias y cinco avanzadas. Los scripts originales y su salida están incluidos en el repositorio. El objetivo es transformar los registros operativos en indicadores útiles para decisiones comerciales.

| Código | Pregunta de negocio | Recursos SQL principales |
|---|---|---|
| Q1 | ¿Qué productos reciben más visualizaciones y de qué marca son? | Agrupación, conteos, ordenamiento y joins. |
| Q2 | ¿Qué categorías facturan más y cuál es su ticket promedio? | Filtro de compras, `GROUP BY`, `HAVING`, agregados. |
| Q3 | ¿Qué marcas convierten mejor sus vistas en compras? | Agregación condicional, porcentajes y joins. |
| Q4 | ¿Cuántos usuarios registrados no presentan actividad? | `LEFT JOIN`, agrupación y filtros. |
| Q5 | ¿Qué cobertura tiene la información de dispositivos? | `LEFT JOIN` y conteos de sesiones. |
| Q6 | ¿En qué etapas del embudo quedan las sesiones? | CTE, `CASE` y agregación por sesión. |
| Q7 | ¿Cuáles son los tres productos más vendidos por categoría? | CTE y `DENSE_RANK`. |
| Q8 | ¿Cómo cambia la actividad minuto a minuto? | `DATE_TRUNC`, `LAG`, sumas acumuladas, media móvil. |
| Q9 | ¿Qué marcas superan el ticket promedio global? | Subconsulta de referencia, `CROSS JOIN`, `HAVING`, `CASE`. |
| Q10 | ¿Qué pasos sigue el usuario antes de comprar? | CTE, `ROW_NUMBER`, `LAG`, `LEAD`. |

## 2.1 Resultados e interpretación

**Q1: productos populares.** El producto 1004856, marca Samsung y categoría `electronics.smartphone`, registra 45 vistas; le siguen productos Samsung y Apple con 38 vistas. Este indicador puede orientar el orden de exhibición del catálogo, pero las visitas no garantizan ventas.

**Q2: facturación por categoría.** Entre las categorías que superan el umbral de la consulta, `electronics.smartphone` registra 20 compras, ingresos de 12,112.96 y ticket promedio de 605.65 unidades monetarias. La siguiente es `electronics.video.tv`, con 9 compras e ingresos de 4,257.89. `HAVING` excluye categorías con una única compra; por ello, las seis filas de salida no corresponden a la totalidad de categorías con venta.

**Q3: conversión por marca.** Para marcas con al menos 20 vistas, Samsung registra 477 vistas y 18 compras (3.77 % según la razón de eventos utilizada), y Apple 364 vistas y 11 compras (3.02 %). Huawei acumula 89 vistas sin compras. Estas tasas son cocientes de eventos y no una tasa de conversión de usuarios únicos.

**Q4: inactividad.** Se identifican 101 usuarios sin eventos entre los 1,113 usuarios cargados. El `LEFT JOIN` preserva los usuarios sin actividad, que desaparecerían de un `INNER JOIN`.

**Q5: cobertura de dispositivos.** La tabla `dispositivos` no tiene registros; por tanto, no se puede analizar el desempeño por tipo de terminal. El reporte también identifica 131 sesiones sin eventos, lo que exige prudencia antes de interpretarlas como abandono.

**Q6: embudo.** De 1,130 sesiones con actividad, 1,078 (95.40 %) solo navegan, 13 (1.15 %) quedan con carrito sin compra y 39 (3.45 %) compran. Los ingresos registrados en sesiones compradoras suman 20,805.68.

**Q7: principales productos por categoría.** Un televisor Samsung (producto 1801881) representa 91.84 % del ingreso de su categoría, mientras que el producto líder de teléfonos inteligentes representa 27.50 %. Esto ilustra distintas concentraciones del ingreso por artículo.

**Q8: evolución temporal.** La consulta genera 32 cortes por minuto, permite observar ingresos acumulados, variaciones respecto del minuto anterior y una media móvil de vistas. El ingreso acumulado termina en 20,805.68.

**Q9: comparación con el ticket global.** Se clasifican marcas cuyo ticket promedio supera al del conjunto total. El análisis permite reconocer segmentos de mayor importe por compra, sin demostrar causalidad sobre la decisión del cliente.

**Q10: secuencia previa a compra.** Las funciones ventana ordenan eventos por sesión e instante y calculan el evento anterior y siguiente. El resultado permite distinguir recorridos cortos de otros con múltiples comparaciones y pasos por el carrito.

### 2.2 Consultas ejecutables

El listado íntegro de las diez consultas se adjunta en el archivo `trabajo_final_parte2.sql`; el **Anexo A** lo incorpora al propio documento para facilitar su evaluación y reproducción.

## 3 Feature Engineering mediante SQL

### 3.1 Objetivo y unidad de análisis

Se diseñaron seis variables por usuario (una fila por `user_id`) con la finalidad de construir un conjunto analítico orientado a la *propensión de compra*. El dataset de entrenamiento demostrativo contiene una etiqueta binaria `compro`, derivada de la presencia de al menos un evento `purchase`. Las variables reflejan solo la ventana observada de 32 minutos, no la actividad total del cliente.

### 3.2 Variables y justificación

| Feature | Definición y justificación | Aplicación potencial en IA |
|---|---|---|
| `vistas_usuario` | Conteo de eventos `view` por usuario. Aproxima intensidad de navegación. | Clasificación de propensión y segmentación por actividad. |
| `productos_distintos_vistos` | Cantidad de `product_id` distintos consultados. Distingue exploración amplia de observación repetida. | Recomendaciones y propensión. |
| `precio_promedio_visto` | Promedio de `price` sobre visualizaciones. Aproxima rango de precios de interés. | Recomendaciones y segmentación de precio. |
| `ratio_carrito_vistas` | Eventos `cart` divididos entre `view`; `NULLIF` evita división por cero. Indicio de intención. | Propensión de compra y seguimiento de carrito. |
| `minutos_activo` | Diferencia en minutos entre primero y último evento observado. Aproxima extensión temporal de la actividad visible. | Propensión y caracterización de sesiones. |

| Feature | Definición y justificación | Aplicación potencial en IA |
|---|---|---|
| `concentracion_marca` | Porcentaje de vistas correspondientes a la marca más vista del usuario. Mide foco en marcas. | Sistemas de recomendación y afinidad de marca. |

### 3.3 Construcción del conjunto analítico

El script `trabajo_final_parte3.sql` contiene una consulta específica por variable (F1–F6), una consulta D1 que reúne las seis con `compro` y una consulta D2 para comparar compradores y no compradores. Se emplean agregaciones, `COUNT(DISTINCT ...)`, `AVG`, `MIN`, `MAX`, expresiones condicionales y subconsultas para la concentración de marca.

### 3.4 Resultados observados

| Métrica promedio | Compradores | No compradores |
|---|---|---|
| Usuarios (conteo) | 39 | 973 |
| Vistas | 5.74 | 3.86 |
| Productos distintos vistos | 2.90 | 2.69 |
| Precio promedio visto | 375.79 | 308.71 |
| Ratio carrito/vistas | 0.174 | 0.009 |
| Minutos activos | 7.86 | 3.30 |
| Concentración de marca | 0.84 | 0.84 |

Tabla 4: Comparación descriptiva a partir de la salida D2 del repositorio.

Los compradores presentan más visualizaciones, mayor relación carrito/vistas y un intervalo observado de actividad más prolongado. La concentración de marca tiene el mismo promedio redondeado en ambos grupos, por lo que no muestra diferenciación descriptiva en esta comparación. Estas diferencias no prueban por sí mismas poder predictivo.

### 3.5 Consideraciones para entrenamiento futuro

El conjunto D1 constituye una demostración de ingeniería de variables, no una evaluación validada de un modelo de aprendizaje automático. Solo 39 de 1,012 usuarios tienen etiqueta positiva (aproximadamente 3.85 %), por lo que existe desbalance de clases. Además, usar toda la ventana para construir predictores de `compro` puede introducir información posterior a la compra (*data leakage*). En una implementación predictiva se deben fijar un instante de corte, un horizonte futuro para la etiqueta y calcular todas las variables exclusivamente con información anterior al corte. Los registros sin vista requieren una política explícita para ratios indefinidos. El ratio de eventos de carrito puede superar uno, ya que no representa una probabilidad y la ventana puede omitir visualizaciones anteriores.

## 4 Optimización de consultas e índices

### 4.1 Diseño de la comparación

Se seleccionaron dos consultas desarrolladas en la sección anterior: Q2 (facturación por categoría) y Q10 (secuencia de eventos antes de una compra). El repositorio incluye resultados de `EXPLAIN` y `EXPLAIN ANALYZE` sobre el esquema de experimentación `tf_p4_original`. Se registraron 15 ejecuciones antes y 15 después de la optimización por consulta, en total 60 mediciones. La

comparación utiliza medianas y promedios de tiempo y se complementa con planes de ejecución completos.

## 4.2 Índices implementados

```sql
CREATE INDEX idx_p4_q2_compras_categoria_usuario
ON tf_p4_original.eventos (category_code, user_id)
INCLUDE (price)
WHERE event_type = 'purchase';

CREATE INDEX idx_p4_q10_sesion_fecha
ON tf_p4_original.eventos
(user_session, event_time, event_id)
INCLUDE (event_type, product_id, price);

CREATE INDEX idx_p4_q10_sesiones_compradoras
ON tf_p4_original.eventos (user_session)
WHERE event_type = 'purchase';
```

El primer índice es parcial y excluye `view` y `cart`: resulta adecuado cuando la consulta agrega únicamente compras. Sus columnas clave ayudan a agrupar por categoría y distinguir compradores, mientras que `INCLUDE(price)` permite cubrir el importe sin añadirlo a la clave de búsqueda. Para Q10, el índice compuesto ordena por sesión y marca temporal, con un desempate por identificador de evento; el índice parcial de sesiones compradoras agiliza identificar sesiones que contienen compras. La selección del optimizador depende del patrón de acceso y de las estadísticas vigentes: crear el índice no garantiza que todos los nodos cambien a acceso por índice.

## 4.3 Comparación de tiempos

| Consulta | Mediana antes (ms) | Mediana después (ms) | Ahorro (ms) | Mejora (%) |
|---|---|---|---|---|
| Q2 | 0.3320 | 0.0520 | 0.2800 | 84.34 |
| Q10 | 1.4190 | 1.1810 | 0.2380 | 16.77 |

Tabla 5: Medianas de tiempo de ejecución, según R2.

La tabla ejecutiva R9 presenta promedios: Q2 disminuye de 0.3495 a 0.0548 ms (84.32 %); Q10 disminuye de 1.4279 a 1.2527 ms (12.27 %). Las cifras difieren de las medianas porque se trata de medidas estadísticas distintas, no de resultados incompatibles.

## 4.4 Comparación de planes: costos, filas y tiempos

| Consulta | Momento | Costo raíz | Filas est. | Filas reales | Ejecución (ms) |
|---|---|---|---|---|---|
| Q2 | Antes | 134.56 | 13 | 6 | 0.321 |
| Q2 | Después | 20.35 | 13 | 6 | 0.050 |
| Q10 | Antes | 289.46 | 40 | 40 | 1.333 |
| Q10 | Después | 175.33 | 40 | 40 | 1.476 |

Tabla 6: Planes de muestra de la repetición 8, según R3. El costo es una unidad interna del planificador.

Para Q2 el acceso a `eventos` cambia de `Seq Scan` (51 filas reales y 4,032 descartadas por filtro; costo estimado 131.16) a `Index Only Scan` (51 filas reales; costo 16.96) utilizando `idx_p4_q2_compras_categoria_usuario`; el nodo indexado muestra 0.007 ms de tiempo

real en la muestra y 0 accesos a heap. El costo total estimado de la consulta cae de 134.56 a 20.35.

Para Q10, el filtrado de sesiones compradoras pasa de `Seq Scan` (4,032 filas descartadas, 51 filas reales; costo 131.04) a `Index Only Scan` con `idx_p4_q10_sesiones_compradoras` (51 filas; costo 16.90). No obstante, un segundo recorrido de los 4,083 eventos continúa con `Seq Scan`. El costo raíz baja de 289.46 a 175.33, pero la repetición 8 tarda 1.476 ms después frente a 1.333 ms antes: una ejecución aislada puede empeorar aunque las medianas de 15 pruebas mejoren.

### 4.5 Planificación y cuellos de botella

En la comparación de medianas, el tiempo de planificación de Q2 sube de 0.0250 a 0.0390 ms, y el de Q10 de 0.2410 a 0.3270 ms. La optimización beneficia principalmente la fase de ejecución, pero añade estructuras que el planificador debe evaluar. En Q2, el cuello de botella inicial era la lectura secuencial y descarte de registros no comprados; el índice parcial evita buena parte de este trabajo. Permanecen operaciones de agregación y ordenamiento.

En Q10, los nodos `Hash Join`, `Sort`, `WindowAgg` e `Incremental Sort` siguen siendo relevantes, porque ordenar y comparar eventos de una sesión no se resuelve exclusivamente con un índice parcial de compras. Las cifras de tiempo de los nodos principales incluyen trabajo de hijos y no deben sumarse entre sí. El índice por sesión y fecha está creado, pero la muestra disponible no demuestra que el optimizador lo haya seleccionado; su existencia no se reporta como mejora comprobada por sí sola.

### 4.6 Alcance y reproducibilidad

El cambio observado es pertinente para la base ensayada, de 4,083 eventos, pero no implica automáticamente mejoras proporcionales en millones de registros. Además, `trabajo_final_parte4.sql` está identificado como script de comparación (R1–R9) y señala que previamente deben ejecutarse scripts 00, 01 y 02. Esos archivos de preparación y captura de mediciones no forman parte del ZIP recibido. Por tanto, los resultados históricos están documentados, pero el equipo debe recuperar esos scripts para reproducir exactamente la secuencia experimental desde cero. Los resultados no se presentan como mediciones realizadas nuevamente para este informe.

## 5 Conclusiones

Los eventos de comercio electrónico permiten construir indicadores de interés comercial a partir de un modelo relacional, especialmente el ingreso por categoría, el embudo de conversión y la trayectoria previa a compra. En el extracto, las ventas se concentran en teléfonos inteligentes y la mayoría de sesiones registradas se limita a navegar.

Se construyeron seis variables de comportamiento orientadas a modelos de propensión de compra. En los datos observados, el ratio carrito/vistas y la extensión de actividad difieren entre compradores y no compradores; sin embargo, se requieren controles temporales y validación para afirmar utilidad predictiva.

Las pruebas documentadas muestran mejoras de mediana de ejecución de 84.34 % para Q2 y 16.77 % para Q10. El beneficio está respaldado por cambios concretos de acceso a datos, aunque la consulta con funciones de ventana conserva operaciones de ordenamiento y recorrido general. Finalmente, la reducida ventana temporal, la falta de información de dispositivos y la ausencia de los scripts de preparación completos condicionan las conclusiones y la reproducibilidad.

# 6 Referencias y archivos fuente


1. Kaggle, *eCommerce behavior data from multi category store*. https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store. Fuente indicada por el README del equipo.

2. Repositorio del equipo, `README.md`, `Trabajo_Final.sql`, archivos de consultas y resultados de las Partes 2, 3 y 4, edición proporcionada del 27 de octubre de 2026.

3. PostgreSQL 16, documentación de sentencias `EXPLAIN`, índices B-tree e índices parciales. https://www.postgresql.org/docs/16/.


# A Código SQL: consultas de negocio (Parte 2)

```sql
-- Trabajo Final - Base de Datos (UTEC)
-- Parte 1, seccion 2: consultas SQL orientadas al negocio
--
-- Esquema: analisis_ecommerce_produccion
-- Datos: eventos de un e-commerce, 2019-11-01 entre 00:00 y 00:31 (4,083 eventos)
--
-- Q1-Q5 consultas basicas e intermedias
-- Q6-Q10 consultas avanzadas
--
-- Nota sobre los datos: el volcado cubre una sola ventana de 32 minutos,
-- por eso el analisis temporal (Q8) se hace a nivel de minuto. Ademas la
-- tabla dispositivos esta vacia y usuarios no trae nombre ni fecha de
-- registro, asi que esas columnas no se usan.

SET search_path TO analisis_ecommerce_produccion;


-- ============================================================================
-- Q1. Que productos atraen mas atencion y de que marca son?
--
-- Resultado: los 15 productos mas vistos. Arriba queda el 1004856 de
-- samsung con 45 vistas, seguido de dos modelos de apple.
--
-- Lectura: la atencion se concentra en pocos smartphones de samsung,
-- apple y xiaomi. Son los que deberian ocupar el home y los primeros
-- resultados del buscador, porque ya tienen la demanda.
-- ============================================================================
SELECT
    e.product_id,
    m.brand_name AS marca,
    COALESCE(NULLIF(e.category_code, ''), 'sin categoria') AS categoria,
    COUNT(*) AS vistas,
    COUNT(DISTINCT e.user_id) AS usuarios,
    ROUND(AVG(e.price), 2) AS precio_promedio
FROM eventos e
INNER JOIN productos p ON p.product_id = e.product_id
INNER JOIN marcas m ON m.brand_id = p.brand_id
WHERE e.event_type = 'view'
GROUP BY e.product_id, m.brand_name, e.category_code
ORDER BY vistas DESC
LIMIT 15;


-- ============================================================================
-- Q2. Que categorias generan ingresos y cual es su ticket promedio?
--
-- Resultado: 6 categorias (el HAVING deja fuera las que vendieron una
-- sola vez). electronics.smartphone factura unos S/ 12,100, mas del
-- doble que la siguiente.
--
-- Lectura: el negocio depende casi por completo de una categoria. El
-- ticket promedio ayuda a decidir donde empujar volumen y donde margen.
-- ============================================================================
```

```sql
SELECT
    COALESCE(NULLIF(category_code, ''), 'sin categoria') AS categoria,
    COUNT(*) AS compras,
    COUNT(DISTINCT user_id) AS compradores,
    ROUND(SUM(price), 2) AS ingreso_total,
    ROUND(AVG(price), 2) AS ticket_promedio,
    MIN(price) AS ticket_minimo,
    MAX(price) AS ticket_maximo
FROM eventos
WHERE event_type = 'purchase'
GROUP BY 1
HAVING COUNT(*) >= 2
ORDER BY ingreso_total DESC;


-- ======================================================================
-- Q3. Que marcas convierten mejor las vistas que reciben?
--
-- Resultado: 28 marcas con 20 vistas o mas. Samsung convierte 3.8% y
-- apple 3.0%; veinte marcas se quedan en cero pese a tener trafico
-- (huawei con 89 vistas es el caso mas llamativo).
--
-- Lectura: una marca muy vista que no vende no tiene problema de
-- trafico sino de precio, stock o ficha de producto. Es la señal mas
-- accionable para el area comercial.
-- ======================================================================
SELECT
    m.brand_name AS marca,
    COUNT(*) FILTER (WHERE e.event_type = 'view') AS vistas,
    COUNT(*) FILTER (WHERE e.event_type = 'cart') AS carritos,
    COUNT(*) FILTER (WHERE e.event_type = 'purchase') AS compras,
    ROUND(100.0 * COUNT(*) FILTER (WHERE e.event_type = 'purchase')
               / NULLIF(COUNT(*) FILTER (WHERE e.event_type = 'view'), 0), 2) AS conversion_pct,
    ROUND(COALESCE(SUM(e.price) FILTER (WHERE e.event_type = 'purchase'), 0), 2) AS ingreso
FROM eventos e
INNER JOIN productos p ON p.product_id = e.product_id
INNER JOIN marcas m ON m.brand_id = p.brand_id
GROUP BY m.brand_name
HAVING COUNT(*) FILTER (WHERE e.event_type = 'view') >= 20
ORDER BY conversion_pct DESC NULLS LAST, vistas DESC;


-- ======================================================================
-- Q4. Cuantos usuarios registrados no hicieron absolutamente nada?
--
-- Resultado: los primeros 20 usuarios ordenados por actividad. Salen
-- todos con cero eventos; en total son 101 de 1,113 usuarios.
--
-- Lectura: aqui el LEFT JOIN es obligatorio. Con INNER JOIN esos 101
-- usuarios desapareceerian y pareceria que toda la base esta activa.
-- Ese 9% inactivo es el publico de una campaña de reactivacion.
-- ======================================================================
SELECT
    u.user_id,
    COUNT(e.event_id) AS eventos,
    COUNT(DISTINCT e.user_session) AS sesiones,
    COUNT(*) FILTER (WHERE e.event_type = 'purchase') AS compras,
    MIN(e.event_time) AS primer_evento,
    MAX(e.event_time) AS ultimo_evento
FROM usuarios u
LEFT JOIN eventos e ON e.user_id = u.user_id
GROUP BY u.user_id
ORDER BY eventos ASC, u.user_id
LIMIT 20;


-- ======================================================================
-- Q5. Que cobertura tiene el registro de dispositivos?
--
-- Resultado: ninguna sesion tiene dispositivo asociado, porque la tabla
-- esta vacia. Tambien aparecen sesiones que no generaron ni un evento
-- (son 131 en total).
--
```

```sql
-- Lectura: hallazgo de calidad de datos. Hoy no se puede segmentar por
-- mobile / desktop / tablet, que es una de las dimensiones mas usadas
-- en e-commerce. Las sesiones sin eventos apuntan a rebote o a un error
-- de tracking.
--
-- Se cuenta con DISTINCT porque dispositivos.user_session no es unica:
-- si una sesion tuviera dos dispositivos, el join duplicaria eventos.
-- ======================================================================
SELECT
    s.user_session,
    s.user_id,
    s.session_start,
    COALESCE(d.device_type, 'no registrado') AS dispositivo,
    COUNT(DISTINCT e.event_id) AS eventos
FROM sesiones s
LEFT JOIN dispositivos d ON d.user_session = s.user_session
LEFT JOIN eventos e ON e.user_session = s.user_session
GROUP BY s.user_session, s.user_id, s.session_start, d.device_type
ORDER BY eventos ASC, s.session_start
LIMIT 20;


-- ======================================================================
-- Q6. Como se reparten las sesiones en el embudo de conversion?
--
-- Resultado: de 1,130 sesiones con actividad, 95% solo navega, 1% deja
-- el carrito abandonado y 3.5% compra. Esas 39 sesiones que compran
-- explican los S/ 20,800 de ingreso.
--
-- Lectura: el embudo es muy angosto. El grupo del medio es el de mayor
-- retorno: ya mostro intencion de compra y solo le falta un empujon
-- (recordatorio, envio gratis, descuento de cierre). Las sesiones que
-- compran ven 4.7 productos en promedio contra 3.5 de las que no.
-- ======================================================================
WITH por_sesion AS (
    SELECT
        user_session,
        COUNT(*) FILTER (WHERE event_type = 'view') AS vistas,
        COUNT(*) FILTER (WHERE event_type = 'cart') AS carritos,
        COUNT(*) FILTER (WHERE event_type = 'purchase') AS compras,
        COALESCE(SUM(price) FILTER (WHERE event_type = 'purchase'), 0) AS ingreso
    FROM eventos
    GROUP BY user_session
),
clasificadas AS (
    SELECT
        por_sesion.*,
        CASE
            WHEN compras > 0 THEN '3. compro'
            WHEN carritos > 0 THEN '2. abandono carrito'
            ELSE '1. solo navego'
        END AS etapa
    FROM por_sesion
)
SELECT
    etapa,
    COUNT(*) AS sesiones,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct,
    ROUND(AVG(vistas), 2) AS vistas_promedio,
    ROUND(SUM(ingreso), 2) AS ingreso
FROM clasificadas
GROUP BY etapa
ORDER BY etapa;


-- ======================================================================
-- Q7. Cuales son los 3 productos top de cada categoria?
--
-- Resultado: 20 filas sobre las 10 categorias con ventas. El contraste
-- es fuerte: electronics.video.tv depende en 92% de un solo televisor
-- samsung, mientras que en smartphones el primer puesto pesa solo 27%.
--
-- Lectura: una categoria que vive de un SKU es fragil, un quiebre de
```

```sql
-- stock la tumba entera. Se usa DENSE_RANK y no ROW_NUMBER para que dos
-- productos empatados compartan puesto.
-- ======================================================================
WITH ventas AS (
    SELECT
        COALESCE(NULLIF(e.category_code, ''), 'sin categoria') AS categoria,
        e.product_id,
        m.brand_name AS marca,
        COUNT(*) AS unidades,
        ROUND(SUM(e.price), 2) AS ingreso
    FROM eventos e
    INNER JOIN productos p ON p.product_id = e.product_id
    INNER JOIN marcas m ON m.brand_id = p.brand_id
    WHERE e.event_type = 'purchase'
    GROUP BY 1, e.product_id, m.brand_name
),
ranking AS (
    SELECT
        ventas.*,
        DENSE_RANK() OVER (PARTITION BY categoria ORDER BY ingreso DESC) AS puesto,
        ROUND(100.0 * ingreso / SUM(ingreso) OVER (PARTITION BY categoria), 2) AS pct_categoria
    FROM ventas
)
SELECT categoria, puesto, product_id, marca, unidades, ingreso, pct_categoria
FROM ranking
WHERE puesto <= 3
ORDER BY categoria, puesto;


-- ======================================================================
-- Q8. Como evoluciona la actividad minuto a minuto?
--
-- Resultado: 32 filas, una por minuto, con el ingreso del minuto, el
-- acumulado, la variacion contra el minuto anterior y una media movil
-- de vistas de 3 minutos. El acumulado cierra en S/ 20,800 y hay 6
-- minutos sin ninguna compra.
--
-- Lectura: el acumulado muestra el ritmo al que entra el dinero y la
-- media movil limpia el ruido para ver si el trafico sube o baja. Los
-- minutos con caida fuerte son los que vale la pena revisar.
--
-- Si mas adelante se cargan varios dias, basta cambiar 'minute' por
-- 'day' en el DATE_TRUNC y la consulta sirve igual.
-- ======================================================================
WITH por_minuto AS (
    SELECT
        DATE_TRUNC('minute', event_time) AS minuto,
        COUNT(*) AS eventos,
        COUNT(*) FILTER (WHERE event_type = 'view') AS vistas,
        COUNT(*) FILTER (WHERE event_type = 'cart') AS carritos,
        COUNT(*) FILTER (WHERE event_type = 'purchase') AS compras,
        COALESCE(SUM(price) FILTER (WHERE event_type = 'purchase'), 0) AS ingreso
    FROM eventos
    GROUP BY 1
)
SELECT
    minuto,
    eventos,
    vistas,
    compras,
    ingreso,
    SUM(ingreso) OVER (ORDER BY minuto) AS ingreso_acumulado,
    LAG(ingreso) OVER (ORDER BY minuto) AS ingreso_anterior,
    ingreso - LAG(ingreso) OVER (ORDER BY minuto) AS variacion,
    ROUND(AVG(vistas) OVER (ORDER BY minuto ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS
        media_movil_vistas
FROM por_minuto
ORDER BY minuto;


-- ======================================================================
-- Q9. Que marcas venden por encima del ticket promedio del negocio?
--
```

```sql
-- Resultado: 5 marcas sobre el promedio general de S/ 408. Apple va
-- primera con un ticket de S/ 924, mas del doble.
--
-- Lectura: estas marcas levantan el ticket promedio del sitio.
-- Empujarlas en recomendaciones sube el ingreso por transaccion sin
-- tener que traer mas trafico, que es la palanca mas cara. El promedio
-- general se calcula una sola vez en el CROSS JOIN y no dentro del
-- SELECT, para no repetir la subconsulta en cada columna.
-- ======================================================================
SELECT
    m.brand_name AS marca,
    COUNT(*) AS compras,
    ROUND(AVG(e.price), 2) AS ticket_promedio,
    ROUND(g.ticket_general, 2) AS ticket_general,
    ROUND(AVG(e.price) - g.ticket_general, 2) AS diferencia,
    CASE
        WHEN AVG(e.price) >= 2.0 * g.ticket_general THEN 'premium'
        WHEN AVG(e.price) >= 1.5 * g.ticket_general THEN 'medio alto'
        ELSE 'sobre el promedio'
    END AS segmento,
    ROUND(SUM(e.price), 2) AS ingreso
FROM eventos e
INNER JOIN productos p ON p.product_id = e.product_id
INNER JOIN marcas m ON m.brand_id = p.brand_id
CROSS JOIN (
    SELECT AVG(price) AS ticket_general
    FROM eventos
    WHERE event_type = 'purchase'
) g
WHERE e.event_type = 'purchase'
GROUP BY m.brand_name, g.ticket_general
HAVING AVG(e.price) > g.ticket_general
ORDER BY ticket_promedio DESC;


-- ======================================================================
-- Q10. Que camino recorre el usuario antes de comprar?
--
-- Resultado: el paso a paso de las sesiones que terminaron en compra,
-- con los segundos que pasaron entre un evento y el siguiente. Se ven
-- dos patrones claros: compras casi inmediatas tras una o dos vistas
-- del mismo producto, y compras pensadas donde el usuario compara
-- varios modelos y pasa por el carrito antes de cerrar.
--
-- Lectura: sirve para saber cuantos productos ve alguien antes de
-- decidirse y si la compra es impulsiva o meditada. Es la base para
-- construir features de comportamiento y tambien para detectar sesiones
-- raras (compras instantaneas sin navegacion previa, posibles bots).
-- ======================================================================
WITH secuencia AS (
    SELECT
        user_session,
        event_time,
        event_type,
        product_id,
        price,
        ROW_NUMBER() OVER (PARTITION BY user_session ORDER BY event_time, event_id) AS paso,
        LAG(event_time) OVER (PARTITION BY user_session ORDER BY event_time, event_id) AS anterior,
        LEAD(event_type) OVER (PARTITION BY user_session ORDER BY event_time, event_id) AS siguiente
    FROM eventos
    WHERE user_session IN (
        SELECT user_session FROM eventos WHERE event_type = 'purchase'
    )
)
SELECT
    user_session,
    paso,
    event_type AS accion,
    product_id,
    price,
    ROUND(EXTRACT(EPOCH FROM (event_time - anterior))::int AS segundos,
    COALESCE(siguiente, 'fin de sesion') AS siguiente_accion,
    CASE
```

```
        WHEN anterior IS NULL THEN 'inicio'
        WHEN EXTRACT(EPOCH FROM (event_time - anterior)) <= 10 THEN 'impulsivo'
        WHEN EXTRACT(EPOCH FROM (event_time - anterior)) <= 60 THEN 'normal'
        ELSE 'pensado'
    END AS ritmo
FROM secuencia
ORDER BY user_session, paso
LIMIT 40;
```

# B Código SQL: Feature Engineering (Parte 3)

```
-- Trabajo Final - Base de Datos (UTEC)
-- Parte 1, seccion 3: features para Machine Learning
--
-- Esquema: analisis_ecommerce_produccion
-- Datos: eventos de un e-commerce (dataset REES46 de Kaggle),
-- 2019-11-01 entre 00:00 y 00:31 (4,083 eventos, 1,012 usuarios)
--
-- Las seis features se calculan por usuario (una fila por user_id).
-- Juntas forman un dataset para un modelo de propension de compra:
-- predecir si un usuario va a comprar a partir de como navega.
--
-- F1 vistas_usuario intensidad de navegacion
-- F2 productos_distintos_vistos exploracion frente a foco
-- F3 precio_promedio_visto segmento de precio que busca
-- F4 ratio_carrito_vistas intencion de compra
-- F5 minutos_activo duracion de la actividad
-- F6 concentracion_marca lealtad a una marca
--
-- Al final, D1 une las seis features con la variable objetivo (compro)
-- en una sola tabla, y D2 compara compradores con no compradores.
--
-- Nota sobre los datos: la ventana es de 32 minutos, por eso las
-- features no usan periodos como "ultimos 30 dias" sino la actividad
-- dentro de la ventana. Las tablas dispositivos y usuarios no traen
-- datos utiles (dispositivos esta vacia y usuarios no tiene nombre ni
-- fecha de registro), asi que todas las features salen de eventos.

SET search_path TO analisis_ecommerce_produccion;


-- ============================================================================
-- F1. vistas_usuario
--
-- Definicion: numero de productos vistos (eventos 'view') por el
-- usuario durante la ventana.
--
-- Justificacion: mide cuanto navega el usuario. Quien ve mas productos
-- esta mas comprometido con la tienda y tiene mas oportunidades de
-- encontrar algo que comprar.
--
-- Uso en IA: variable de entrada de un modelo de propension de compra
-- y de segmentacion (clustering) de usuarios por nivel de actividad.
--
-- Resultado: el usuario mas activo vio 44 productos en 32 minutos. En
-- promedio, los compradores ven 5.7 productos y los no compradores 3.9.
-- ============================================================================
SELECT
    user_id,
    COUNT(*) FILTER (WHERE event_type = 'view') AS vistas_usuario
FROM eventos
GROUP BY user_id
ORDER BY vistas_usuario DESC
LIMIT 10;


-- ============================================================================
-- F2. productos_distintos_vistos
--
-- Definicion: cantidad de productos diferentes que vio el usuario.
```

```sql
--
-- Justificacion: distingue dos comportamientos con las mismas vistas.
-- Quien ve 10 veces el mismo producto esta decidiendo una compra; quien
-- ve 10 productos distintos esta explorando. Comparada con F1 indica
-- si el usuario ya tiene un producto en mente.
--
-- Uso en IA: propension de compra y sistemas de recomendacion (a un
-- usuario explorador conviene mostrarle variedad; a uno enfocado,
-- alternativas del mismo producto).
--
-- Resultado: hay usuarios con 25 vistas y 25 productos distintos (solo
-- exploran) y compradores con 11 vistas de un solo producto.
-- ======================================================================
SELECT
    user_id,
    COUNT(DISTINCT product_id) FILTER (WHERE event_type = 'view') AS productos_distintos_vistos
FROM eventos
GROUP BY user_id
ORDER BY productos_distintos_vistos DESC
LIMIT 10;


-- ======================================================================
-- F3. precio_promedio_visto
--
-- Definicion: precio promedio de los productos que vio el usuario.
--
-- Justificacion: aproxima el segmento de precio y el poder adquisitivo
-- del usuario. Quien mira productos de S/ 1,000 no responde igual a
-- una oferta que quien mira productos de S/ 50.
--
-- Uso en IA: segmentacion de clientes, recomendaciones dentro del
-- rango de precio del usuario y personalizacion de descuentos.
--
-- Resultado: los usuarios que miran productos mas caros llegan a un
-- promedio de S/ 2,011. Los compradores miran en promedio productos de
-- S/ 376 y los no compradores de S/ 309.
-- ======================================================================
SELECT
    user_id,
    ROUND(AVG(price) FILTER (WHERE event_type = 'view'), 2) AS precio_promedio_visto
FROM eventos
GROUP BY user_id
HAVING COUNT(*) FILTER (WHERE event_type = 'view') > 0
ORDER BY precio_promedio_visto DESC
LIMIT 10;


-- ======================================================================
-- F4. ratio_carrito_vistas
--
-- Definicion: agregados al carrito divididos entre productos vistos.
--
-- Justificacion: es la señal de intencion mas fuerte antes de la
-- compra. Un ratio alto indica que el usuario no solo mira sino que
-- selecciona. NULLIF evita dividir entre cero en usuarios sin vistas.
--
-- Uso en IA: variable clave de la propension de compra y disparador de
-- acciones en tiempo real (por ejemplo, un recordatorio a quien agrego
-- al carrito y no compro).
--
-- Resultado: es la feature que mas separa a los grupos: 0.174 en
-- compradores frente a 0.009 en no compradores (unas 19 veces mas).
-- Algunos usuarios tienen ratio mayor que 1 porque agregaron al carrito
-- un producto que habian visto antes de las 00:00, fuera de la ventana.
-- ======================================================================
SELECT
    user_id,
    COUNT(*) FILTER (WHERE event_type = 'cart') AS carritos,
    COUNT(*) FILTER (WHERE event_type = 'view') AS vistas,
    ROUND(COUNT(*) FILTER (WHERE event_type = 'cart')::numeric
          / NULLIF(COUNT(*) FILTER (WHERE event_type = 'view'), 0), 3) AS ratio_carrito_vistas
FROM eventos
```

```sql
GROUP BY user_id
HAVING COUNT(*) FILTER (WHERE event_type = 'cart') > 0
ORDER BY ratio_carrito_vistas DESC NULLS LAST
LIMIT 10;


-- ======================================================================
-- F5. minutos_activo
--
-- Definicion: minutos entre el primer y el ultimo evento del usuario.
--
-- Justificacion: mide cuanto tiempo dedica el usuario a la tienda. Un
-- usuario que se queda varios minutos esta evaluando opciones; uno con
-- 0 minutos hizo una sola accion y se fue (rebote).
--
-- Uso en IA: propension de compra y deteccion de rebote. En un modelo
-- de fraude tambien sirve: muchas acciones en muy poco tiempo son
-- tipicas de un bot.
--
-- Resultado: los compradores pasan 7.9 minutos activos en promedio y
-- los no compradores 3.3. Varios usuarios estuvieron activos casi toda
-- la ventana (31 minutos).
-- ======================================================================
SELECT
    user_id,
    MIN(event_time) AS primer_evento,
    MAX(event_time) AS ultimo_evento,
    ROUND(EXTRACT(EPOCH FROM MAX(event_time) - MIN(event_time)) / 60, 2) AS minutos_activo
FROM eventos
GROUP BY user_id
ORDER BY minutos_activo DESC
LIMIT 10;


-- ======================================================================
-- F6. concentracion_marca
--
-- Definicion: proporcion de las vistas del usuario que corresponden a
-- su marca mas vista. Va de 0 a 1: 1 significa que solo miro una marca.
--
-- Justificacion: separa a los usuarios leales a una marca de los que
-- comparan entre marcas. Se calcula en dos pasos: primero las vistas
-- por usuario y marca (CTE), luego la marca con mas vistas sobre el
-- total del usuario.
--
-- Uso en IA: recomendacion (a un usuario leal se le recomiendan
-- productos de su marca) y segmentacion por lealtad.
--
-- Resultado: el promedio es 0.84 en los dos grupos, asi que no ayuda a
-- predecir la compra. Si sirve para recomendar: hay usuarios con 21
-- vistas de una sola marca.
-- ======================================================================
WITH vistas_por_marca AS (
    SELECT
        e.user_id,
        m.brand_name,
        COUNT(*) AS vistas
    FROM eventos e
    INNER JOIN productos p ON p.product_id = e.product_id
    INNER JOIN marcas m ON m.brand_id = p.brand_id
    WHERE e.event_type = 'view'
    GROUP BY e.user_id, m.brand_name
)
SELECT
    user_id,
    COUNT(*) AS marcas_vistas,
    SUM(vistas) AS vistas_totales,
    ROUND(MAX(vistas)::numeric / SUM(vistas), 2) AS concentracion_marca
FROM vistas_por_marca
GROUP BY user_id
HAVING SUM(vistas) >= 5
ORDER BY concentracion_marca DESC, vistas_totales DESC
LIMIT 10;
```

```sql

-- ======================================================================
-- D1. Dataset final: las seis features y la variable objetivo
--
-- Una fila por usuario, lista para entrenar un modelo. La variable
-- objetivo compro vale 1 si el usuario hizo al menos una compra.
-- Los usuarios sin vistas quedan con 0 o NULL en las features de
-- vistas, que es lo que el modelo deberia recibir.
-- ======================================================================
WITH actividad AS (
    SELECT
        user_id,
        COUNT(*) FILTER (WHERE event_type = 'view') AS vistas_usuario,
        COUNT(DISTINCT product_id) FILTER (WHERE event_type = 'view') AS productos_distintos_vistos,
        ROUND(AVG(price) FILTER (WHERE event_type = 'view'), 2) AS precio_promedio_visto,
        ROUND(COUNT(*) FILTER (WHERE event_type = 'cart')::numeric
              / NULLIF(COUNT(*) FILTER (WHERE event_type = 'view'), 0), 3) AS ratio_carrito_vistas,
        ROUND(EXTRACT(EPOCH FROM MAX(event_time) - MIN(event_time)) / 60, 2) AS minutos_activo,
        MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS compro
    FROM eventos
    GROUP BY user_id
),
vistas_por_marca AS (
    SELECT e.user_id, m.brand_name, COUNT(*) AS vistas
    FROM eventos e
    INNER JOIN productos p ON p.product_id = e.product_id
    INNER JOIN marcas m ON m.brand_id = p.brand_id
    WHERE e.event_type = 'view'
    GROUP BY e.user_id, m.brand_name
),
marca AS (
    SELECT user_id, ROUND(MAX(vistas)::numeric / SUM(vistas), 2) AS concentracion_marca
    FROM vistas_por_marca
    GROUP BY user_id
)
SELECT
    a.user_id,
    a.vistas_usuario,
    a.productos_distintos_vistos,
    a.precio_promedio_visto,
    a.ratio_carrito_vistas,
    a.minutos_activo,
    m.concentracion_marca,
    a.compro
FROM actividad a
LEFT JOIN marca m ON m.user_id = a.user_id
ORDER BY a.compro DESC, a.vistas_usuario DESC
LIMIT 15;


-- ======================================================================
-- D2. Compradores frente a no compradores
--
-- Promedio de cada feature en los dos grupos. Si una feature tiene
-- valores muy distintos entre quienes compran y quienes no, es una
-- buena candidata para predecir la compra.
--
-- Resultado: 39 compradores y 973 no compradores. El ratio de carrito
-- (0.174 frente a 0.009), los minutos activos (7.9 frente a 3.3) y las
-- vistas (5.7 frente a 3.9) separan bien a los grupos. La concentracion
-- de marca no (0.84 en ambos).
--
-- Lectura: con solo 39 compradores el dataset esta muy desbalanceado
-- (3.9% de positivos). Un modelo tendria que usar pesos por clase o
-- sobremuestreo, y evaluarse con recall o AUC, no con exactitud.
-- Ademas, en produccion las features deben calcularse solo con los
-- eventos anteriores a la compra, para no filtrar la respuesta.
-- ======================================================================
WITH actividad AS (
    SELECT
        user_id,
        COUNT(*) FILTER (WHERE event_type = 'view') AS vistas_usuario,
```

```sql
        COUNT(DISTINCT product_id) FILTER (WHERE event_type = 'view') AS productos_distintos_vistos,
        AVG(price) FILTER (WHERE event_type = 'view') AS precio_promedio_visto,
        COUNT(*) FILTER (WHERE event_type = 'cart')::numeric
            / NULLIF(COUNT(*) FILTER (WHERE event_type = 'view'), 0) AS ratio_carrito_vistas,
        EXTRACT(EPOCH FROM MAX(event_time) - MIN(event_time)) / 60 AS minutos_activo,
        MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS compro
    FROM eventos
    GROUP BY user_id
),
vistas_por_marca AS (
    SELECT e.user_id, m.brand_name, COUNT(*) AS vistas
    FROM eventos e
    INNER JOIN productos p ON p.product_id = e.product_id
    INNER JOIN marcas m ON m.brand_id = p.brand_id
    WHERE e.event_type = 'view'
    GROUP BY e.user_id, m.brand_name
),
marca AS (
    SELECT user_id, MAX(vistas)::numeric / SUM(vistas) AS concentracion_marca
    FROM vistas_por_marca
    GROUP BY user_id
)
SELECT
    CASE a.compro WHEN 1 THEN 'compradores' ELSE 'no compradores' END AS grupo,
    COUNT(*) AS usuarios,
    ROUND(AVG(a.vistas_usuario), 2) AS vistas_usuario,
    ROUND(AVG(a.productos_distintos_vistos), 2) AS productos_distintos_vistos,
    ROUND(AVG(a.precio_promedio_visto), 2) AS precio_promedio_visto,
    ROUND(AVG(a.ratio_carrito_vistas), 3) AS ratio_carrito_vistas,
    ROUND(AVG(a.minutos_activo), 2) AS minutos_activo,
    ROUND(AVG(m.concentracion_marca), 2) AS concentracion_marca
FROM actividad a
LEFT JOIN marca m ON m.user_id = a.user_id
GROUP BY a.compro
ORDER BY a.compro DESC;
```

# C Código SQL: análisis de planes e índices (Parte 4)

**Nota:** el siguiente archivo de comparaciones requiere scripts de preparación anteriores que no están incluidos en la entrega suministrada. Los comandos de creación de índices pertinentes se reprodujeron en la sección 4.

```sql

-- ============================================================
-- TRABAJO FINAL - BASE DE DATOS
-- PUNTO 4: OPTIMIZACION DE CONSULTAS E INDICES
-- ARCHIVO: 03_COMPARACION_Y_PLANES.sql
-- ============================================================
-- R1: Numero de mediciones
-- R2: Comparacion detallada de tiempos
-- R3: Costos, filas y tiempos
-- R4: Tipos de Scan e indices
-- R5: Operaciones mas costosas
-- R6: Validacion de datos y consultas
-- R7: Verificacion de indices
-- R8: Planes completos
-- R9: Tabla ejecutiva ANTES vs DESPUES
--
-- IMPORTANTE:
-- Ejecutar previamente los scripts 00, 01 y 02.
-- Este archivo solo consulta resultados; no modifica datos.
-- ============================================================

SET search_path TO tf_p4_original, public;


-- ============================================================
-- R1. NUMERO DE MEDICIONES
-- Deben existir 15 mediciones por consulta y etapa.
```

```sql
-- Total esperado: 60 mediciones.
-- ============================================================

SELECT
    consulta,
    etapa,
    COUNT(*) AS mediciones
FROM tf_p4_original.mediciones
GROUP BY consulta, etapa
ORDER BY consulta, etapa;


-- ============================================================
-- R2. COMPARACION DETALLADA DE TIEMPOS
-- Mediana y promedio de ejecucion y planificacion.
-- Una mejora positiva indica reduccion del tiempo.
-- ============================================================

WITH resumen AS (
    SELECT
        consulta,
        etapa,

        ROUND(
            (
                percentile_cont(0.5)
                WITHIN GROUP (
                    ORDER BY (
                        plan_real #>> '{0,Execution Time}'
                    )::numeric
                )
            )::numeric, 4
        ) AS ejec_mediana_ms,

        ROUND(
            AVG(
                (plan_real #>>
                 '{0,Execution Time}')::numeric
            ), 4
        ) AS ejec_promedio_ms,

        ROUND(
            (
                percentile_cont(0.5)
                WITHIN GROUP (
                    ORDER BY (
                        plan_real #>> '{0,Planning Time}'
                    )::numeric
                )
            )::numeric, 4
        ) AS plan_mediana_ms,

        ROUND(
            AVG(
                (plan_real #>>
                 '{0,Planning Time}')::numeric
            ), 4
        ) AS plan_promedio_ms,

        COUNT(*) AS n

    FROM tf_p4_original.mediciones
    GROUP BY consulta, etapa
)

SELECT
    a.consulta,
    a.n AS repeticiones_antes,
    d.n AS repeticiones_despues,

    a.ejec_mediana_ms AS ejec_antes_ms,
    d.ejec_mediana_ms AS ejec_despues_ms,

```

```
    ROUND(
        a.ejec_mediana_ms - d.ejec_mediana_ms, 4
    ) AS ahorro_ms,

    ROUND(
        100 * (
            a.ejec_mediana_ms - d.ejec_mediana_ms
        ) / NULLIF(a.ejec_mediana_ms, 0),
        2
    ) AS mejora_pct,

    a.plan_mediana_ms AS planning_antes_ms,
    d.plan_mediana_ms AS planning_despues_ms,

    a.ejec_promedio_ms AS ejec_promedio_antes_ms,
    d.ejec_promedio_ms AS ejec_promedio_despues_ms

FROM resumen a
JOIN resumen d
    ON a.consulta = d.consulta

WHERE a.etapa = 'ANTES'
  AND d.etapa = 'DESPUES'

ORDER BY a.consulta;


-- ============================================================
-- R3. COSTOS, FILAS Y TIEMPOS
-- Se usa la repeticion 8 como muestra central de 15.
-- ============================================================

SELECT
    consulta,
    etapa,

    plan_estimado #>>
        '{0,Plan,Node Type}' AS nodo_raiz,

    (
        plan_estimado #>>
        '{0,Plan,Total Cost}'
    )::numeric AS costo_estimado,

    (
        plan_estimado #>>
        '{0,Plan,Plan Rows}'
    )::bigint AS filas_estimadas,

    (
        plan_real #>>
        '{0,Plan,Actual Rows}'
    )::bigint AS filas_reales,

    (
        plan_real #>>
        '{0,Planning Time}'
    )::numeric AS planificacion_ms,

    (
        plan_real #>>
        '{0,Execution Time}'
    )::numeric AS ejecucion_ms

FROM tf_p4_original.mediciones

WHERE repeticion = 8

ORDER BY consulta, etapa;


-- ============================================================
-- R4. TIPOS DE SCAN E INDICES UTILIZADOS
```

```sql
-- Lee todos los nodos del plan de ejecucion.
-- ============================================================

WITH RECURSIVE nodos AS (

    SELECT
        m.consulta,
        m.etapa,
        m.plan_real #> '{0,Plan}' AS nodo,
        0 AS nivel

    FROM tf_p4_original.mediciones m
    WHERE m.repeticion = 8

    UNION ALL

    SELECT
        n.consulta,
        n.etapa,
        hijo.value,
        n.nivel + 1

    FROM nodos n

    CROSS JOIN LATERAL
        jsonb_array_elements(
            COALESCE(
                n.nodo->'Plans',
                '[]'::jsonb
            )
        ) AS hijo(value)
)

SELECT
    consulta,
    etapa,
    nivel,

    nodo->>'Node Type' AS tipo_scan,
    nodo->>'Relation Name' AS tabla,

    COALESCE(
        nodo->>'Index Name',
        'NINGUNO'
    ) AS indice,

    (
        nodo->>'Total Cost'
    )::numeric AS costo_estimado,

    (
        nodo->>'Plan Rows'
    )::bigint AS filas_estimadas,

    (
        nodo->>'Actual Rows'
    )::bigint AS filas_reales_por_loop,

    (
        nodo->>'Actual Loops'
    )::bigint AS loops,

    COALESCE(
        (
            nodo->>'Rows Removed by Filter'
        )::bigint,
        0
    ) AS filas_descartadas_por_loop,

    (
        nodo->>'Actual Total Time'
    )::numeric AS tiempo_real_ms_por_loop

```

```
FROM nodos

WHERE nodo->>'Node Type' LIKE '%Scan%'

ORDER BY consulta, etapa, nivel;


-- ============================================================
-- R5. OPERACIONES MAS COSTOSAS
-- Muestra las 8 operaciones de mayor tiempo acumulado
-- por consulta y etapa.
--
-- IMPORTANTE:
-- Actual Total Time incluye el trabajo de nodos hijos.
-- Los tiempos de los nodos no deben sumarse.
-- ============================================================

WITH RECURSIVE nodos AS (

    SELECT
        m.consulta,
        m.etapa,
        m.plan_real #> '{0,Plan}' AS nodo,
        0 AS nivel

    FROM tf_p4_original.mediciones m
    WHERE m.repeticion = 8

    UNION ALL

    SELECT
        n.consulta,
        n.etapa,
        hijo.value,
        n.nivel + 1

    FROM nodos n

    CROSS JOIN LATERAL
        jsonb_array_elements(
            COALESCE(
                n.nodo->'Plans',
                '[]'::jsonb
            )
        ) AS hijo(value)
),

ordenados AS (

    SELECT
        consulta,
        etapa,
        nivel,

        nodo->>'Node Type' AS operacion,

        COALESCE(
            nodo->>'Index Name',
            ''
        ) AS indice,

        (
            nodo->>'Total Cost'
        )::numeric AS costo_estimado,

        (
            nodo->>'Actual Total Time'
        )::numeric AS tiempo_por_loop_ms,

        (
            nodo->>'Actual Rows'
        )::bigint AS filas_reales_por_loop,

```

```
        (
            nodo->>'Actual Loops'
        )::bigint AS loops,

        ROW_NUMBER() OVER (
            PARTITION BY consulta, etapa

            ORDER BY (
                nodo->>'Actual Total Time'
            )::numeric DESC NULLS LAST
        ) AS prioridad

    FROM nodos
)

SELECT
    consulta,
    etapa,
    nivel,
    operacion,
    indice,
    costo_estimado,
    tiempo_por_loop_ms,
    filas_reales_por_loop,
    loops

FROM ordenados

WHERE prioridad <= 8

ORDER BY consulta, etapa, prioridad;


-- ============================================================
-- R6. VALIDACION DE LA BASE DE DATOS
-- Verifica cantidad de eventos, compras y sesiones
-- compradoras.
-- ============================================================

SELECT
    COUNT(*) AS eventos,

    COUNT(*) FILTER (
        WHERE event_type = 'purchase'
    ) AS compras,

    COUNT(DISTINCT user_session) FILTER (
        WHERE event_type = 'purchase'
    ) AS sesiones_compradoras

FROM tf_p4_original.eventos;


-- Validacion de las consultas originales.
-- Se utiliza MD5 para verificar su contenido.

SELECT
    codigo,
    md5(sql_original) AS huella_consulta

FROM tf_p4_original.consultas

ORDER BY codigo;


-- ============================================================
-- R7. INDICES EXISTENTES DESPUES DE LA OPTIMIZACION
-- ============================================================

SELECT
    indexname,
    indexdef

```

```sql
FROM pg_indexes

WHERE schemaname = 'tf_p4_original'
  AND tablename = 'eventos'

ORDER BY indexname;


-- ============================================================
-- R8. PLANES COMPLETOS EXPLAIN Y EXPLAIN ANALYZE
-- Se utiliza la repeticion 8.
-- ============================================================

SELECT
    consulta,
    etapa,
    plan_estimado,
    plan_real

FROM tf_p4_original.mediciones

WHERE repeticion = 8

ORDER BY consulta, etapa;


-- ============================================================
-- R9. TABLA EJECUTIVA DE COMPARACION DE TIEMPOS
-- CONSULTAS Q2 Y Q10
-- ANTES VS DESPUES
--
-- ESTA DEBE SER LA ULTIMA CONSULTA DEL ARCHIVO
--
-- Presenta:
-- 1. Numero de pruebas
-- 2. Tiempo ANTES
-- 3. Tiempo DESPUES
-- 4. Ahorro en milisegundos
-- 5. Porcentaje de mejora
-- 6. Tiempos de planificacion
-- 7. Resultado de la optimizacion
--
-- Mejora positiva: DESPUES tarda menos.
-- Mejora negativa: DESPUES tarda mas.
-- ============================================================

WITH resumen AS (

    SELECT
        consulta,

        COUNT(*) FILTER (
            WHERE etapa = 'ANTES'
        ) AS repeticiones_antes,

        COUNT(*) FILTER (
            WHERE etapa = 'DESPUES'
        ) AS repeticiones_despues,

        AVG(
            (
                plan_real #>>
                '{0,Execution Time}'
            )::numeric
        ) FILTER (
            WHERE etapa = 'ANTES'
        ) AS ejecucion_antes_ms,

        AVG(
            (
                plan_real #>>
                '{0,Execution Time}'
            )::numeric
```

```
        ) FILTER (
            WHERE etapa = 'DESPUES'
        ) AS ejecucion_despues_ms,

        AVG(
            (
                plan_real #>>
                '{0,Planning Time}'
            )::numeric
        ) FILTER (
            WHERE etapa = 'ANTES'
        ) AS planning_antes_ms,

        AVG(
            (
                plan_real #>>
                '{0,Planning Time}'
            )::numeric
        ) FILTER (
            WHERE etapa = 'DESPUES'
        ) AS planning_despues_ms

    FROM tf_p4_original.mediciones

    GROUP BY consulta
)

SELECT
    c.consulta,

    COALESCE(
        r.repeticiones_antes, 0
    ) AS pruebas_antes,

    COALESCE(
        r.repeticiones_despues, 0
    ) AS pruebas_despues,

    ROUND(
        r.ejecucion_antes_ms, 4
    ) AS tiempo_antes_ms,

    ROUND(
        r.ejecucion_despues_ms, 4
    ) AS tiempo_despues_ms,

    ROUND(
        r.ejecucion_antes_ms -
        r.ejecucion_despues_ms, 4
    ) AS ahorro_ms,

    ROUND(
        100 * (
            r.ejecucion_antes_ms -
            r.ejecucion_despues_ms
        ) / NULLIF(
            r.ejecucion_antes_ms, 0
        ),
        2
    ) AS mejora_pct,

    ROUND(
        r.planning_antes_ms, 4
    ) AS planificacion_antes_ms,

    ROUND(
        r.planning_despues_ms, 4
    ) AS planificacion_despues_ms,

    CASE

        WHEN COALESCE(
            r.repeticiones_antes, 0
```

```
        ) = 0

        OR COALESCE(
           r.repeticiones_despues, 0
        ) = 0

        THEN 'FALTAN MEDICIONES'

        WHEN r.ejecucion_despues_ms
             < r.ejecucion_antes_ms

        THEN 'MEJORA'

        WHEN r.ejecucion_despues_ms
             > r.ejecucion_antes_ms

        THEN 'NO MEJORA'

        ELSE 'SIN CAMBIO'

    END AS resultado

FROM (
    VALUES ('Q2'), ('Q10')
) AS c(consulta)

LEFT JOIN resumen AS r
    ON r.consulta = c.consulta

ORDER BY
    CASE c.consulta
        WHEN 'Q2' THEN 1
        WHEN 'Q10' THEN 2
        ELSE 3
    END;
```